

Joc Snake pe ESP32 cu modul afisaj SSD1306, 4 butoane si buzzer

Studenti:

Popescu Andrei-Cristian

Pătru Theodora

Disciplina: Proiect Informatica Aplicata

Bucuresti 2024

Cuprins

Componente necesare	3
Descriere joc	3
Diagrama de asamblare si maparea pinilor	3
Poze mod legare placuta-ecran-butoane-buzzer	5
Codul pentru joc.....	6
Concluzii si directii posibile	12

Componente necesare

Pentru acest proiect sunt necesare:

1. o placuta Esp32
2. modul afisaj SSD1306
3. 4 butoane
4. un buzzer
5. fire de legatura
6. doua breadboard

Descriere joc

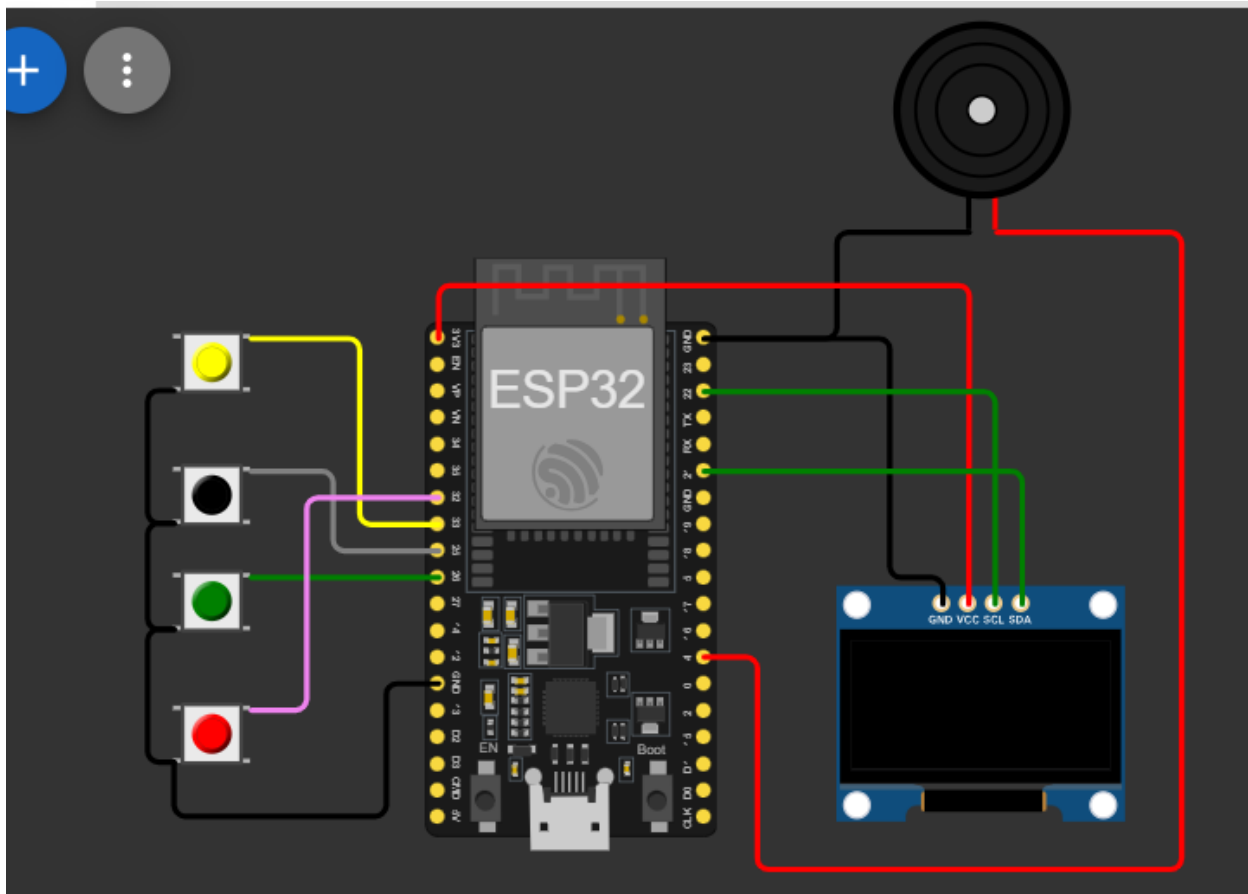
Proiectul este o adaptare a jocului Snake Game. Acesta are ca scop colectarea cat mai multor fructe cu sarpele fara ca acesta sa se auto-interescteze sau sa loveasca peretii.

Sarpele poate fi miscat sus, dreapta, stanga, jos prin apasarea butonelor corespunzatoare (vezi codul sau tabelul de mapare). Buzzerul are trei sunete diferite: unul pentru inceputul jocului, unul pentru momentul in care sarpele manaca fructul si unul pentru momentul de coliziune (adica pierdere).

Diagrama de asamblare si maparea pinilor

Pentru a avea o imagine clara a conexiunilor placuta-ecran-butoane-buzzer, putem folosi diagrama si tabelul urmatoar:

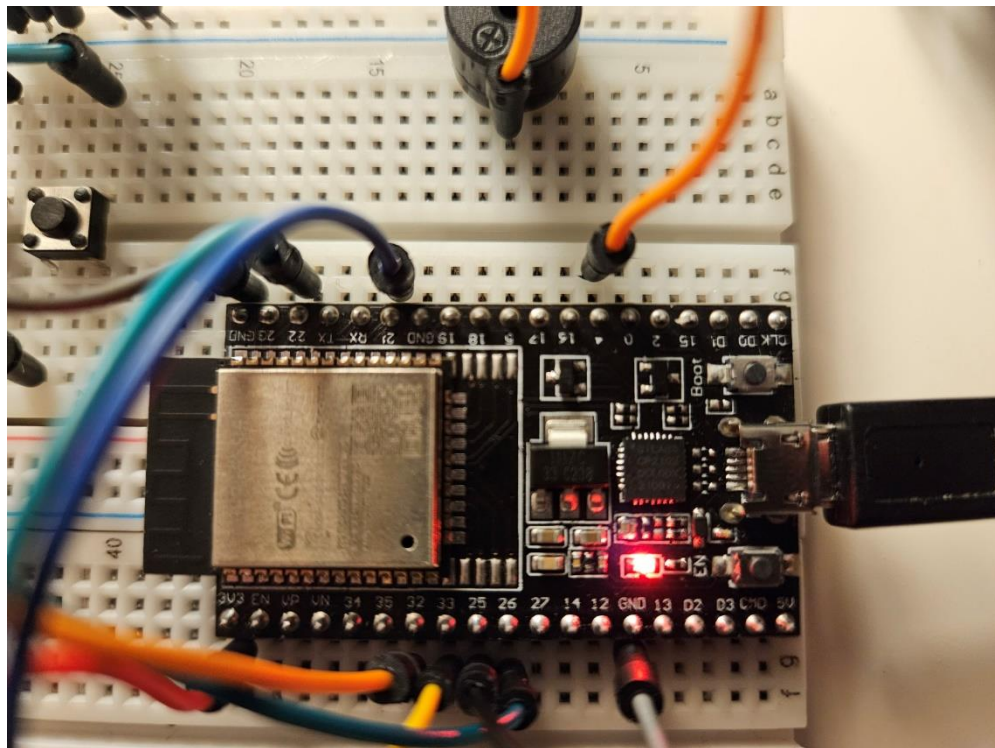
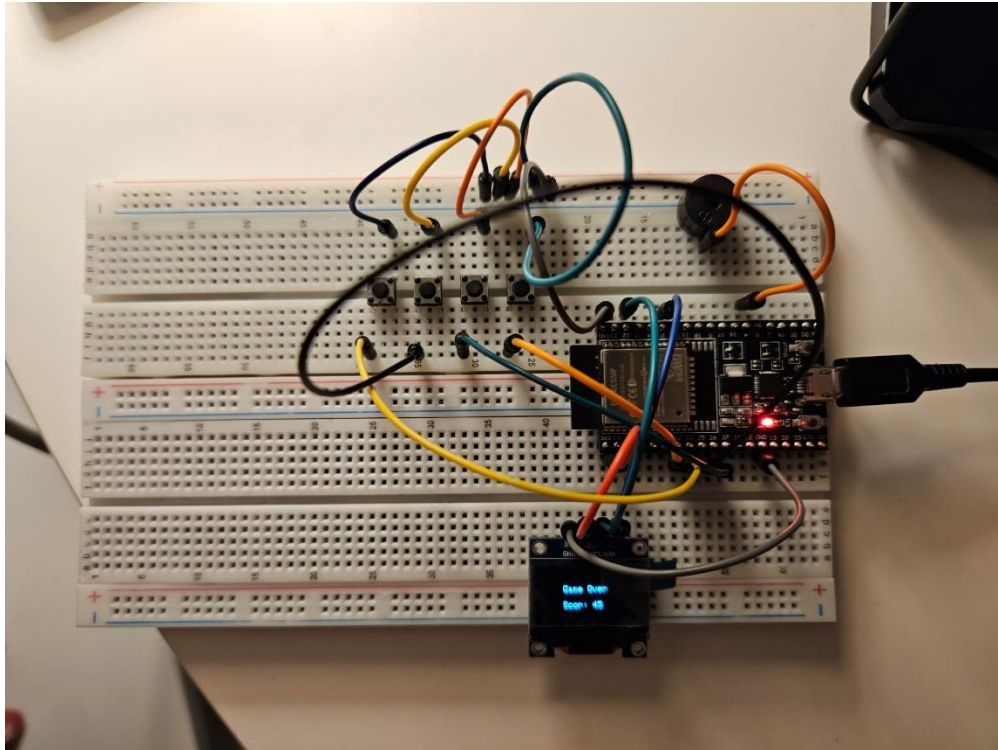
Pin ESP32	Componenta/Pin componenta
GND	GND – ecran, butoane, buzzer
21	SDA - ecran
22	SCL - ecran
3V3	VCC - ecran
33	buton - sus
32	buton - jos
26	buton - stanga
25	buton - dreapta
4	buzzer

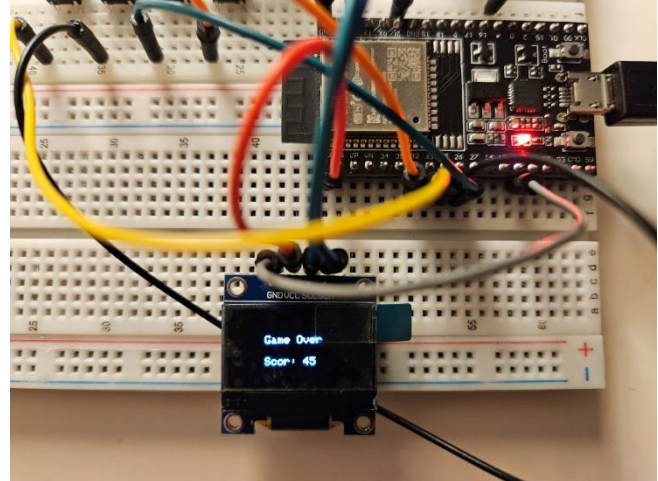
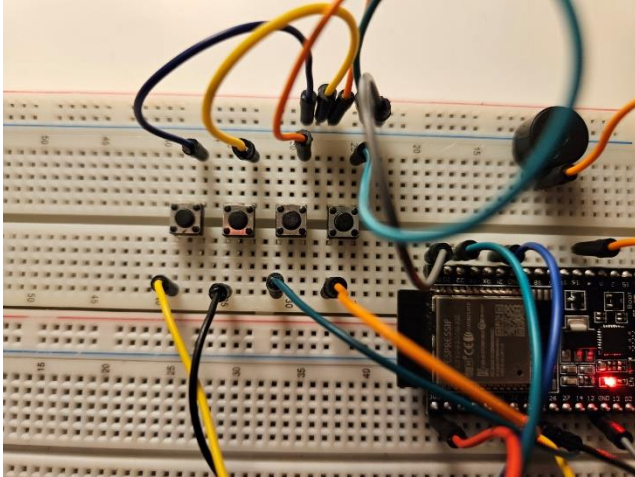


Modul de legare este prezentat in Wokwi si poate fi accesat la link-ul:

<https://wokwi.com/projects/397617296176380929>

Poze mod legare placuta-ecran-butoane-buzzer





Codul pentru joc

Codul este scris integral de noi, nu este adaptat niciunui model existent pe internet.

Codul este scris folosind Arduino IDE. Pentru a functiona codul, este necesara adaugarea librariilor *Adafruit_GFX.h* si *Adafruit_SSD1306.h*.

Explicatiile generale asupra functiilor folosite sunt prezentate ca si comentarii in interiorul codului acolo unde este necesar.

```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_RESET -1
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

#define BUTTON_UP 33
#define BUTTON_DOWN 32
#define BUTTON_LEFT 26
#define BUTTON_RIGHT 25

#define BUZZER_PIN 4 // Pinul pentru buzzer

// Definirea tonurilor pentru buzzer
#define TONE_START 1000 // Frecventa sunetului de start
#define TONE_LOSE 1000 // Frecventa sunetului de pierdere
#define TONE_EAT 1500 // Frecventa sunetului de consumare a hranei
```

```

// Durata sunetelor
#define SOUND_DURATION 100 // Durata fiecarui sunet in milisecunde

// Starile sunetelor
#define SOUND_NONE 0
#define SOUND_START 1
#define SOUND_LOSE 2
#define SOUND_EAT 3

int buzzerState = SOUND_NONE; // Starea actuala a sunetului

// Proprietatile sarpelui
int *snakeX;
int *snakeY;
int snakeLength = 1;
int snakeDirection = 0; // 0: dreapta, 1: sus, 2: stanga, 3: jos

// Proprietatile hranei
int foodX, foodY;

// Scorul
int scor = 0;

void setup() {
    Serial.begin(115200);

    // Initializarea ecranului
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println(F("Alocarea SSD1306 a esuat"));
        for (;;);
    }
    // Initializarea butoanelor
    pinMode(BUTTON_UP, INPUT_PULLUP);
    pinMode(BUTTON_DOWN, INPUT_PULLUP);
    pinMode(BUTTON_LEFT, INPUT_PULLUP);
    pinMode(BUTTON_RIGHT, INPUT_PULLUP);

    // Initializarea buzzer-ului
    pinMode(BUZZER_PIN, OUTPUT);

    // Initializarea hranei
    randomSeed(random() * 313 + 4567);

    // Initializarea jocului
    initializeGame();
}

```

```

}

void loop() {
    handleInput();

    // Mutarea sarpelui
    moveSnake();

    // Verificarea coliziunilor
    Collisions();

    // Generarea hranei daca a fost consumata
    checkFood();

    // Desenarea jocului
    drawGame();

    // Actualizarea scorului
    displayScor();

    // Redarea sunetului pentru buzzer daca este necesar
    playBuzzer();

    delay(100);
}

void initializeGame() {
    // Alocarea memoriei pentru sarpe
    snakeX = new int[100];
    snakeY = new int[100];

    // Initializarea sarpelui
    snakeX[0] = SCREEN_WIDTH / 2;
    snakeY[0] = SCREEN_HEIGHT / 2;

    // Initializarea hranei
    generateFood();

    // Initializarea scorului
    scor = 0;

    // Redarea sunetului de start
    buzzerState = SOUND_START;
}

```



```

void generateFood() {
    // Generarea pozitiei hranei
    foodX = random(1, SCREEN_WIDTH - 5);
    foodY = random(1, SCREEN_HEIGHT - 5);
}

void handleInput() {

    // Modificarea directiei sarpelui
    if (digitalRead(BUTTON_UP) == LOW && snakeDirection != 3) {
        snakeDirection = 1;
    } else if (digitalRead(BUTTON_DOWN) == LOW && snakeDirection != 1) {
        snakeDirection = 3;
    } else if (digitalRead(BUTTON_LEFT) == LOW && snakeDirection != 0) {
        snakeDirection = 2;
    } else if (digitalRead(BUTTON_RIGHT) == LOW && snakeDirection != 2) {
        snakeDirection = 0;
    }
}

void moveSnake() {
    // Miscarea corpului sarpelui
    for (int i = snakeLength - 1; i > 0; i--) {
        snakeX[i] = snakeX[i - 1];
        snakeY[i] = snakeY[i - 1];
    }

    // Miscarea capului sarpelui
    switch (snakeDirection) {
        case 0: snakeX[0]++; break; // Dreapta
        case 1: snakeY[0]--; break; // Sus
        case 2: snakeX[0]--; break; // Stanga
        case 3: snakeY[0]++; break; // Jos
    }
}

void Collisions() {
    // Verificarea coliziunilor cu marginile ecranului
    if (snakeX[0] >= SCREEN_WIDTH || snakeX[0] < 0 || snakeY[0] >= SCREEN_HEIGHT
    || snakeY[0] < 0) {
        buzzerState = SOUND_LOSE;
        gameOver();
    }

    // Verificarea coliziunilor cu propriul corp

```

```

    for (int i = 1; i < snakeLength; i++) {
        if (snakeX[0] == snakeX[i] && snakeY[0] == snakeY[i]) {
            buzzerState = SOUND_LOSE;
            gameOver();
        }
    }
}

void checkFood() {
    if (snakeX[0] == foodX && snakeY[0] == foodY) {
        // Cresterea lungimii sarpelui
        snakeLength++;

        // Cresterea scorului
        scor += 5;

        // Generarea unei noi bucati de mancare
        generateFood();

        // Redarea sunetului pentru consumarea hranei
        buzzerState = SOUND_EAT;
    }
}

void drawGame() {
    // stergerea ecranului
    display.clearDisplay();

    // Desenarea marginilor ecranului
    display.drawRect(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);

    // Desenarea sarpelui
    for (int i = 0; i < snakeLength; i++) {
        display.fillRect(snakeX[i], snakeY[i], 4, 4, SSD1306_WHITE);
    }

    // Desenarea hranei
    display.fillRect(foodX, foodY, 4, 4, SSD1306_WHITE);

    // Actualizarea ecranului
    display.display();
}

void displayScor() {
    // Afisarea scorului pe ecran
    display.setTextSize(1);

```

```

    display.setTextColor(SSD1306_WHITE);
    display.setCursor(2, 1.5);
    display.print("Scor: ");
    display.println(scor);
    display.display();
}

void gameOver() {
    // Afisarea ecranului de sfarsit de joc
    display.clearDisplay();
    display.setTextSize(1.5);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(SCREEN_WIDTH / 4, SCREEN_HEIGHT / 3);
    display.println("Game Over");
    display.setTextSize(1.5);
    display.setCursor(SCREEN_WIDTH / 4, (SCREEN_HEIGHT / 3) + 20);
    display.print("Scor: ");
    display.println(scor);
    display.display();
    buzzerState = SOUND_LOSE;
    playBuzzer();

    // Asteptarea inceperii unui nou joc
    while (true) {
        if (digitalRead(BUTTON_UP) == LOW) {
            // Initierea unui nou joc
            initializeGame();
            return;
        }
        delay(100);
    }
}

void playBuzzer() {
    // Redarea sunetului corespunzator starii buzzer-ului
    switch (buzzerState) {
        case SOUND_START:
            tone(BUZZER_PIN, TONE_START, SOUND_DURATION);
            delay(SOUND_DURATION);
            noTone(BUZZER_PIN);
            buzzerState = SOUND_NONE;
            break;
        case SOUND_LOSE:
            tone(BUZZER_PIN, 262, 200);
            delay(200);
    }
}

```

```
    tone(BUZZER_PIN, 294, 200);  
    delay(200);  
    tone(BUZZER_PIN, 330, 200);  
    delay(200);  
    noTone(BUZZER_PIN);  
    buzzerState = SOUND_NONE;  
    break;  
case SOUND_EAT:  
    tone(BUZZER_PIN, TONE_EAT, SOUND_DURATION);  
    delay(SOUND_DURATION);  
    noTone(BUZZER_PIN);  
    buzzerState = SOUND_NONE;  
    break;  
default:  
    break;  
}  
}
```

Concluzii si directii posibile

Prototipul creat de noi este unul de baza si poate si dezvoltat in viitor prin adaugarea unor niveluri de viteza (sarpele sa aiba o viteza mai mare sau mai mica, depinde de preferintele utilizatorului), adaugarea unei interefete prietenoase, adaugarea unei clasament sau adaugarea unor niveluri de dificultatea.